



Information Technology
Generalist

PROG1400 – Final Game Project Deliverables

Weight: 35% of Final Grade

Total Points: 35 (Analytic Rubric)

The Final Game Project is the culmination of your semester-long iterative work in PROG1400. You will demonstrate your understanding of **object-oriented programming**, **UML modelling**, **complexity management**, **Python implementation**, and **refactoring/debugging strategies** by delivering a polished, working game built using Python and Pygame.

Your submission must show evidence of **learning across the entire term**, from initial modelling, to implementation, to continuous refinement.

Required Deliverables

You must submit the following items to Brightspace to receive full consideration under the Final Project Rubric:

1. Final Working Game (Python + Pygame)

Your game must:

- Run without errors
- Use OOP architecture aligned with your UML
- Demonstrate encapsulation, abstraction, inheritance, and polymorphism
- Show meaningful design beyond the baseline template
- Include collision handling, game flow logic, and restart/retry functionality
- Represent a finished or near-finished version of your project

You must upload:

- Your full project folder (zipped)
- All **.py** files
- Assets (images, audio) your game depends on
- Any external libraries (other than Pygame) must be declared in a README

2. Updated UML Class Diagram (UML v3)

Your final UML must:

- Reflect the real architecture of your completed game
- Use correct UML notation (attributes, methods, relationships)
- Show separation of concerns (engine, entities, game state, UI, etc.)
- Demonstrate intentional architectural choices
- Build upon and refine your earlier UML versions (v1 and v2)

The UML can be submitted as:

- A PNG/JPG
 - A PDF
 - An exported image from Visual Paradigm, Draw.io, Lucidchart, etc.
-

3. GitHub Repository Link

Your GitHub repo must:

- Contain your most recent project version
- Include meaningful commit history (evidence of iterative growth)
- Demonstrate proper file organization
- Contain a README that includes:
 - Game description
 - Controls
 - Features implemented
 - Known bugs (if any)
 - Credits for assets or libraries

This supports PROG1400's emphasis on **professional workflow** and **portfolio development**.

4. Final Reflection (500–750 Words)

Your written reflection must address the following:

A. Key Design Decisions

Describe at least three major decisions you made while designing your game's architecture (e.g., hierarchy, inheritance strategy, collision model, game loop structure). Explain **why** you made these choices.

B. Biggest Debugging Challenge

Describe the most difficult bug or issue you encountered. Include:

- What the problem looked like
- How you diagnosed it
- The strategies you used (print debugging, refactoring, UML updates, modularization, etc.)
- What you learned from resolving it

C. OOP Principle You Improved the Most

Explain which principle—encapsulation, abstraction, inheritance, or polymorphism—strengthened the most in your project and why. Reference specific classes or systems.

D. What You Would Improve With More Time

Identify realistic improvements such as:

- Better architecture
- Cleaner separation of UI logic
- Additional states or enemies
- Performance enhancements
- More refined design patterns

This reflection demonstrates **metacognition**, growth, and the ability to evaluate your own learning—key goals of PROG1400’s scaffolded, PBL-based structure.

 Alignment With Course Philosophy

This final project:

- Integrates **UML modelling, Python OOP, debugging, and iterative refinement**
- Builds directly on the **low-stakes scaffolding** of earlier assignments
- Encourages **creative ownership** by allowing students to pick their game concept
- Reinforces learning through a **single evolving project**, not isolated tasks
- Matches program goals of **design literacy, problem-solving, and practical implementation**

 How Deliverables Map to the Final Rubric

Deliverable	Rubric Category	LO Alignment
Final Game	Implementation, Debugging, Enhancements	LO2, LO3, LO4
UML Diagram	Architecture & OOP Design	LO1
GitHub Repo	Organization, Professional Presentation	LO3
Final Reflection	Enhancements & Presentation	LO2, LO3, LO4

Total: **35 points** → **35% of course grade**

 What an A-Level Final Submission Looks Like

An A-level project (Level 4 across categories):

- Has a **clean, intentional architecture** with strong OOP separation
- Uses inheritance and polymorphism effectively

- Demonstrates thoughtful modularity and avoids large monolithic code blocks
- Includes evidence of **refactoring and debugging discipline**
- Shows meaningful enhancements beyond a basic implementation
- Uses GitHub professionally
- Includes a **polished, insightful reflection**
- Feels “portfolio-ready”